

MC521 - Relatório X

Júlio da Silva Telles

22 de Junho de 2026

1 Dijkstra? (CodeForces - 20C)

O problema pede explicitamente para encontrar o caminho mais curto entre o vértice 1 e o vértice N em um grafo ponderado e não-direcionado. O diferencial prático é que não basta apenas calcular a distância final: é necessário recuperar e imprimir a sequência exata de vértices que compõem esse caminho.

1.1 Solução

A solução baseia-se no Algoritmo de Dijkstra aliado a um vetor de rastreamento de predecessores para permitir o *backtracking* da rota. A complexidade de tempo atinge $\mathcal{O}(M \log N)$. O fluxo funciona da seguinte maneira:

1. **Exploração do Grafo:** Utilizamos o Dijkstra padrão auxiliado por uma fila de prioridades (`priority_queue`) orientada a valores mínimos (`greater<ii>`), extraindo sempre o nó com a menor distância acumulada.
2. **Rastreamento (Predecessores):** Criamos um vetor `pred` inicializado com -1 . Sempre que encontramos um caminho mais curto para um nó vizinho (`v.first`) passando pelo nó atual (`u`), atualizamos a distância mínima e salvamos a informação de que "viemos de u " fazendo `pred[v.first] = u`.
3. **Reconstrução do Caminho:** Após o fim do algoritmo, checamos se o vértice N foi alcançado (`dist[n] != INF`). Se sim, utilizamos a função recursiva `print_path` para percorrer o vetor `pred` de trás para frente (começando em N e indo até o 1). A recursão garante que os nós sejam impressos na ordem correta da origem ao destino. Se N for inalcançável, imprime-se -1 .

2 Road Construction (Aizu - 2249)

O problema descreve a necessidade de reconstruir as estradas de um país partindo da capital (cidade 1) para todas as outras cidades, mantendo a distância mínima original entre a capital e cada cidade. Como cada estrada possui um custo de construção, o objetivo é selecionar um subconjunto de estradas que satisfaça a condição das distâncias mínimas e, ao mesmo tempo, minimize o custo total da obra.

2.1 Solução

Este problema exige uma variação do Algoritmo de Dijkstra para encontrar a Árvore de Caminhos Mínimos (*Shortest Path Tree*), ponderando pelo custo em caso de empates. A complexidade de tempo é de $\mathcal{O}(M \log N)$, onde M é o número de estradas e N o número de cidades. O algoritmo segue os passos:

1. **Cálculo das Distâncias Mínimas:** Primeiro, executamos o algoritmo de Dijkstra tradicional partindo da capital (nó 1) utilizando exclusivamente as *distâncias* como peso das arestas. Isso preenche o vetor `dist` com a distância mínima exata da capital para cada cidade do país.
2. **Otimização de Custos (Tie-breaking):** Após conhecer a distância ótima para cada cidade, precisamos escolher qual estrada construir para conectá-la à malha. Iteramos sobre cada cidade i (de 2 até N) e analisamos todas as suas estradas adjacentes.
3. **Seleção da Aresta:** Para cada vizinho v , verificamos se a estrada pertence a um caminho mínimo válido através da equação `dist[v] + peso == dist[i]`. Caso seja uma estrada válida, atualizamos o menor custo de construção necessário para conectar a cidade i (`min_cost`). Por fim, somamos esse menor custo à variável `total`.